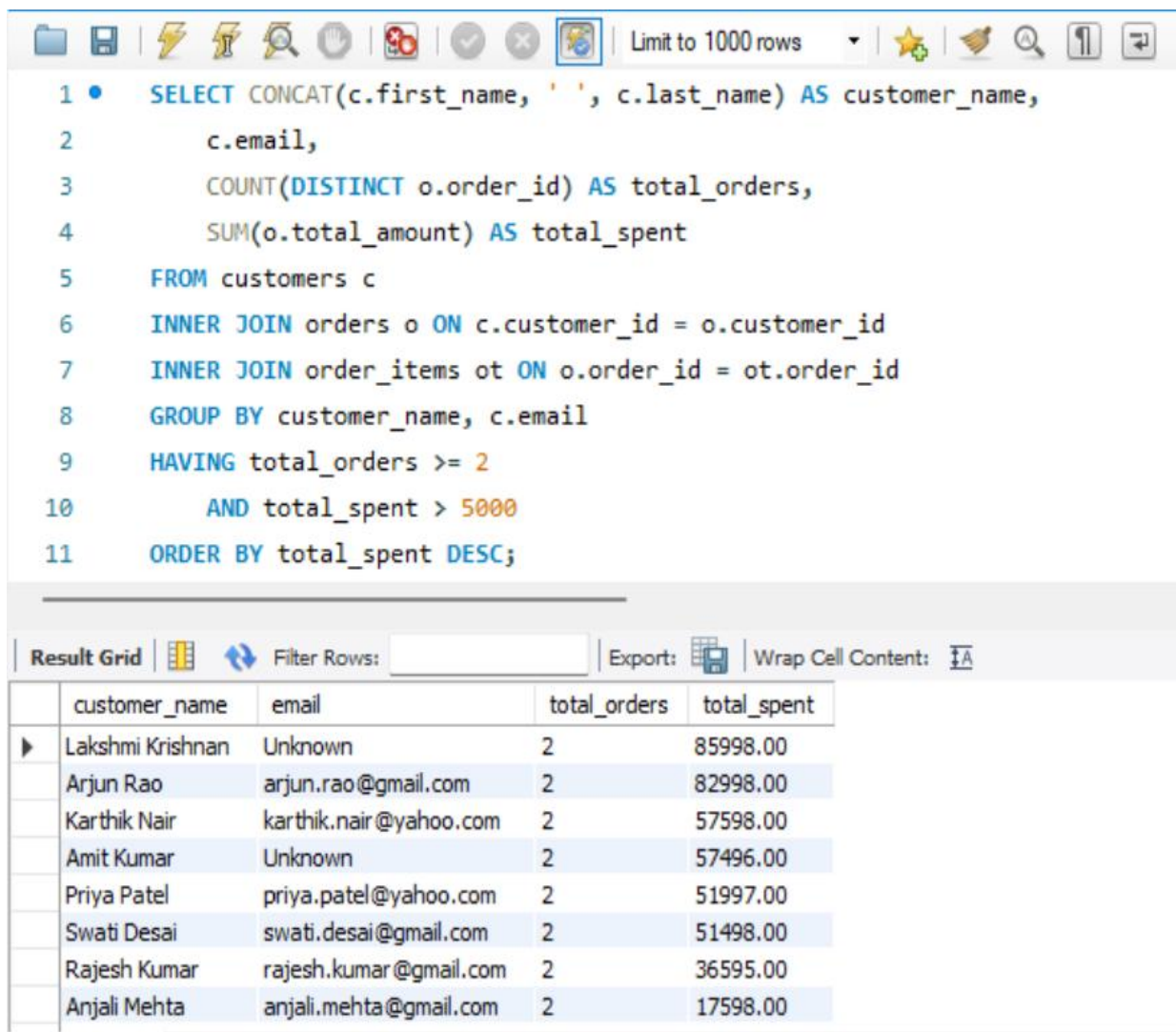


RESULTS OF SQL QUERIES

PART 1- ETL Pipeline

- **Query 1: Customer Purchase History**

Generate a detailed report showing each customer's name, email, total number of orders placed, and total amount spent. Include only customers who have placed at least 2 orders and spent more than ₹5,000. Order by total amount spent in descending order.



The screenshot shows a SQL query editor interface. The query is as follows:

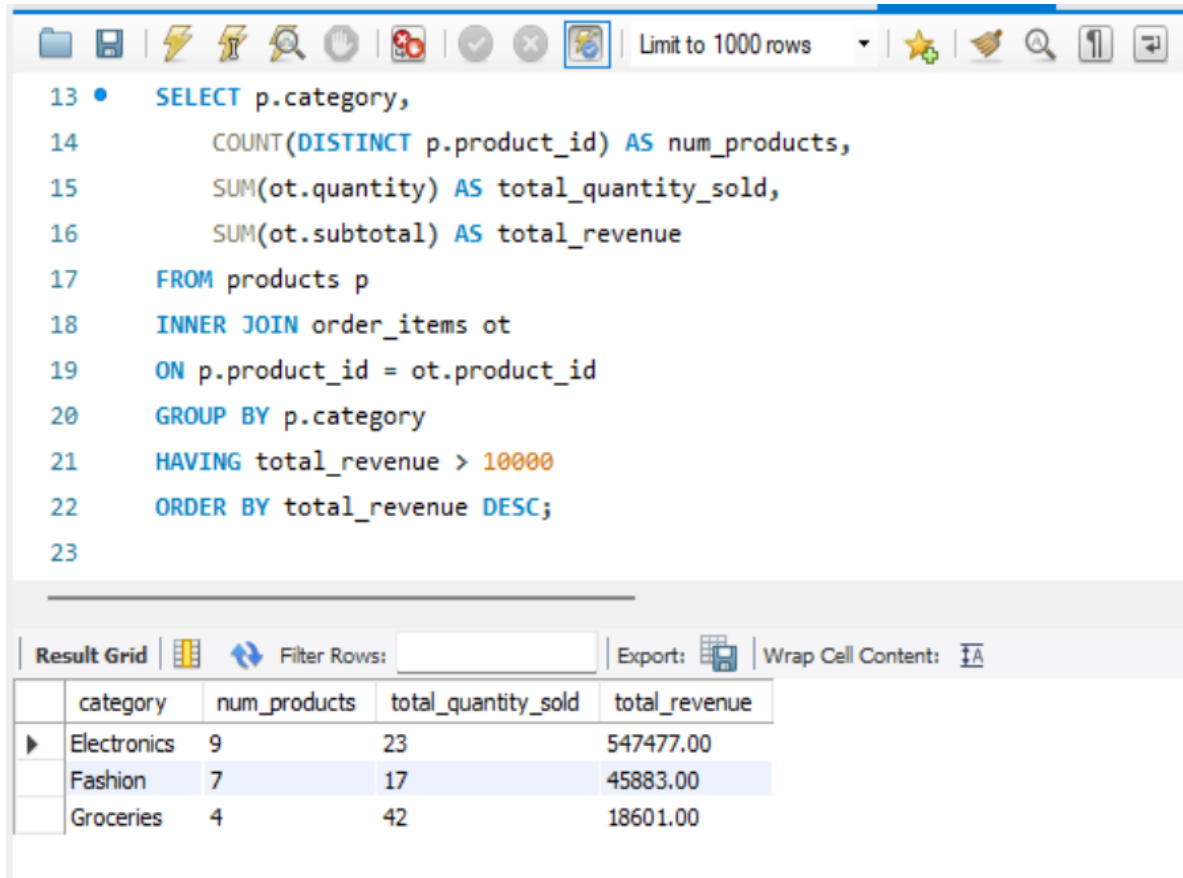
```
1 • SELECT CONCAT(c.first_name, ' ', c.last_name) AS customer_name,  
2     c.email,  
3     COUNT(DISTINCT o.order_id) AS total_orders,  
4     SUM(o.total_amount) AS total_spent  
5 FROM customers c  
6 INNER JOIN orders o ON c.customer_id = o.customer_id  
7 INNER JOIN order_items oi ON o.order_id = oi.order_id  
8 GROUP BY customer_name, c.email  
9 HAVING total_orders >= 2  
10    AND total_spent > 5000  
11 ORDER BY total_spent DESC;
```

Below the query editor is the 'Result Grid' showing the output of the query. The grid has columns for customer_name, email, total_orders, and total_spent. The results are sorted by total_spent in descending order.

customer_name	email	total_orders	total_spent
Lakshmi Krishnan	Unknown	2	85998.00
Arjun Rao	arjun.rao@gmail.com	2	82998.00
Karthik Nair	karthik.nair@yahoo.com	2	57598.00
Amit Kumar	Unknown	2	57496.00
Priya Patel	priya.patel@yahoo.com	2	51997.00
Swati Desai	swati.desai@gmail.com	2	51498.00
Rajesh Kumar	rajesh.kumar@gmail.com	2	36595.00
Anjali Mehta	anjali.mehta@gmail.com	2	17598.00

- **Query 2: Product Sales Analysis**

For each product category, show the category name, number of different products sold, total quantity sold, and total revenue generated. Only include categories that have generated more than ₹10,000 in revenue. Order by total revenue descending.



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
13 • SELECT p.category,  
14     COUNT(DISTINCT p.product_id) AS num_products,  
15     SUM(ot.quantity) AS total_quantity_sold,  
16     SUM(ot.subtotal) AS total_revenue  
17 FROM products p  
18 INNER JOIN order_items ot  
19 ON p.product_id = ot.product_id  
20 GROUP BY p.category  
21 HAVING total_revenue > 10000  
22 ORDER BY total_revenue DESC;  
23
```

Below the query editor is a toolbar with options like "Result Grid", "Filter Rows", "Export", and "Wrap Cell Content". The results are displayed in a table with the following data:

	category	num_products	total_quantity_sold	total_revenue
▶	Electronics	9	23	547477.00
	Fashion	7	17	45883.00
	Groceries	4	42	18601.00

- **Query 3: Monthly Sales Trend**

Show monthly sales trends for the year 2024. For each month, display the month name, total number of orders, total revenue, and the running total of revenue (cumulative revenue from January to that month)

```
26 • SELECT
27     month_name,
28     total_orders,
29     monthly_revenue,
30     SUM(monthly_revenue) OVER (ORDER BY month_num) AS cumulative_revenue
31 FROM (
32     SELECT
33         MONTH(o.order_date) AS month_num,
34         MONTHNAME(o.order_date) AS month_name,
35         COUNT(DISTINCT o.order_id) AS total_orders,
36         SUM(o.total_amount) AS monthly_revenue
37     FROM orders o
38     WHERE YEAR(o.order_date) = 2024
39     GROUP BY month_num, month_name
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	month_name	total_orders	monthly_revenue	cumulative_revenue
▶	January	7	128542.00	128542.00
	February	11	225732.00	354274.00
	March	11	172736.00	527010.00
	April	6	84951.00	611961.00

PART 3 - Star Schema

- **Query 1: Monthly Sales Drill-Down**

The CEO wants to see sales performance broken down by time periods. Start with yearly total, then quarterly, then monthly sales for 2024.

```
1 • SELECT
2     dd.year,
3     dd.quarter,
4     dd.month_name,
5     SUM(fs.total_amount) AS total_amount,
6     SUM(fs.quantity_sold) AS total_quantity
7 FROM fact_sales fs
8 INNER JOIN dim_date dd
9 ON fs.date_key = dd.date_key
10 WHERE dd.year = 2024
11 GROUP BY dd.year, dd.quarter, dd.month_name
12 ORDER BY dd.month_name;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	year	quarter	month_name	total_amount	total_quantity
▶	2024	Q1	February	225582.10	31
	2024	Q1	January	128542.00	11

- **Query 2: Top 10 Products by Revenue**

The product manager needs to identify top-performing products. Show the top 10 products by revenue, along with their category, total units sold, and revenue contribution percentage.

```
14 • SELECT
15     dp.product_name,
16     dp.category,
17     SUM(fs.quantity_sold) AS units_sold,
18     SUM(fs.total_amount) AS revenue,
19     ROUND(SUM(fs.total_amount)/SUM(SUM(fs.total_amount)) OVER() * 100, 2) AS revenue_percentage
20 FROM fact_sales fs
21 INNER JOIN dim_product dp
22 ON fs.product_key = dp.product_key
23 GROUP BY dp.product_name, dp.category
24 ORDER BY revenue DESC
25 LIMIT 10;
```

Result Grid

  Filter Rows:

Export: 

Wrap Cell Content: 

	product_name	category	units_sold	revenue	revenue_percentage
▶	iPhone 13	Electronics	1	69999.00	19.77
	HP Laptop	Electronics	1	52999.00	14.97
	Apple MacBook Pro	Electronics	1	52999.00	14.97
	Samsung Galaxy S21	Electronics	1	45999.00	12.99
	OnePlus Nord	Electronics	1	45999.00	12.99
	Samsung TV 43inch	Electronics	1	32999.00	9.32
	Dell Monitor 24inch	Electronics	1	12999.00	3.67

- **Query 3: Customer Segmentation**

Marketing wants to target high-value customers. Segment customers into 'High Value' (>₹50,000 spent), 'Medium Value' (₹20,000-₹50,000), and 'Low Value' (<₹20,000). Show count of customers and total revenue in each segment.

```

27 • WITH customer_totals AS (
28     SELECT
29         dc.customer_key,
30         SUM(fs.total_amount) AS total_spent
31     FROM fact_sales fs
32     JOIN dim_customer dc ON fs.customer_key = dc.customer_key
33     GROUP BY dc.customer_key
34 )
35 SELECT
36     CASE
37         WHEN total_spent > 50000 THEN 'High Value'
38         WHEN total_spent BETWEEN 20000 AND 50000 THEN 'Medium Value'
39         ELSE 'Low Value'
40     END AS customer_segment,
    customer_count,
    total_revenue,
    avg_revenue_per_customer
FROM customer_totals

```

customer_segment	customer_count	total_revenue	avg_revenue_per_customer
High Value	3	175997.00	58665.666667
Medium Value	3	124997.00	41665.666667
Low Value	12	53130.10	4427.508333